

NOTE À L'ATTENTION DE LA DIRECTION

Analyse de la dette technique

Audit du code source — projet École 2.0 (React/TypeScript, Supabase) — 28 669 lignes analysées

Résumé exécutif

École 2.0 repose sur des fondations plus saines qu'il n'y paraît au premier abord : mode TypeScript strict activé, politiques RLS Supabase documentées et vérifiées (rapport de migration du 2 juillet 2026), et une checklist d'audit UI à 40 points déjà en place. Ce n'est pas un projet livré sans discipline.

L'analyse du code source (28 669 lignes TypeScript) fait toutefois apparaître neuf points de dette technique concrets, dont un constat qui mérite l'attention immédiate de la direction : les modules Facturation et Paiements ne disposent d'aucun test automatisé. Le reste relève d'un mélange de nettoyage à faible coût (dépendance inutilisée, licence manquante) et de deux chantiers structurants déjà présents sur la feuille de route produit.

La demande de cette note : réserver un créneau dédié (de l'ordre d'un sprint court) pour traiter les quatre « quick wins » identifiés, en priorité la sécurisation par des tests des parcours financiers.

Contexte et méthode

Cette analyse s'appuie sur un examen direct du code source (archive complète du dépôt fournie), pas seulement sur la documentation. Ont été vérifiés : la taille et la complexité des fichiers, la présence de tests, la configuration TypeScript et CI, les dépendances réellement utilisées (par recherche d'imports, pas seulement par lecture du package.json), et les traces de secrets ou de marqueurs de dette (TODO, any, @ts-ignore). L'installation des dépendances n'étant pas disponible dans cet environnement, certains outils (ESLint, tsc, détection de duplication) n'ont pas pu être exécutés directement — voir section « Pour affiner le chiffrage ».

Constats principaux

Constat	Catégorie	Impact	Effort	Priorité
Modules Facturation et Paiements sans aucun test	Produit/Finance	Élevé	Moyen	1
Aucun lint ni typecheck en CI (mode strict jamais vérifié)	Qualité	Élevé	Faible	1
Dépendance MUI totalement inutilisée (0 import réel)	Qualité	Faible	Faible	1
Licence de projet non formalisée	Juridique	Moyen	Faible	1
Couverture de tests faible (9 fichiers pour 50+ composants)	Qualité	Moyen	Moyen	2
Clé Supabase anon codée en dur (déjà connue en interne, repère « P2.3 »)	Sécurité	Moyen	Faible	2
5 « god components » de plus de 800 lignes (ElevesScreen.tsx : 2570)	Architecture	Élevé	Élevé	3
Performances mobiles non optimisées	Produit	Élevé	Moyen	2

Constat	Catégorie	Impact	Effort	Priorité
Mode offline non livré (cœur de la mission produit)	Produit	Élevé	Élevé	3

Priorité 1 = quick win (à traiter sous 2 semaines) · Priorité 2 = à planifier ce trimestre · Priorité 3 = chantier structurant, à séquencer.

Plan d'action recommandé

Phase 1 — Quick wins (1 sprint, faible effort)

- Écrire des tests sur BillingWorkspace.tsx et PaymentsWorkspace.tsx (130 et 193 lignes) avant toute nouvelle évolution de ces écrans.
- Ajouter un job « typecheck » (tsc --noEmit) et un job ESLint dans quality-gates.yml — le mode strict existe déjà dans tsconfig.json, il ne manque que la vérification automatique.
- Retirer @mui/material, @mui/icons-material, @emotion/react et @emotion/styled du package.json (aucun import réel détecté) et supprimer leur chunk dans vite.config.ts.
- Ajouter un fichier LICENSE explicite avant toute diffusion publique élargie.

Phase 2 — Chantiers à planifier (ce trimestre)

- Remplacer utils/supabase/info.tsx par des variables d'environnement (VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY) — le code source documente déjà la marche à suivre en commentaire.
- Étendre la couverture de tests aux modules admin, notifications et programme, aujourd'hui sans test dédié.
- Cadrer les performances mobiles, un enjeu directement lié à la mission du produit (enseignants en zones à connectivité limitée).

Phase 3 — Chantiers structurants (à séquencer)

- Découper les 5 fichiers de plus de 800 lignes (ElevesScreen.tsx en tête, à 2570 lignes) en sous-composants testables — commencer par extraire la logique métier des hooks/services.
- Livrer le mode offline prévu en feuille de route — techniquement plus lourd, mais central pour l'usage terrain.

Pour affiner le chiffrage (prochaine étape)

Cette analyse a déjà vérifié le code réel (tailles de fichiers, imports, tests présents, configuration). Il reste, une fois pnpm install exécuté en local, à faire tourner :

- `pnpm audit` pour objectiver les vulnérabilités de dépendances (non exécutable dans cet environnement, sans accès réseau).
- `tsc --noEmit` pour lister précisément les erreurs de typage que le build actuel n'affiche pas.
- `vitest run --coverage` pour un pourcentage de couverture exact, module par module.
- Un outil de détection de duplication (ex. jscpd) sur les 5 gros fichiers identifiés, pour quantifier ce qui peut être factorisé.

Conclusion

Aucun de ces constats ne remet en cause la viabilité du projet : le mode strict TypeScript est actif, les politiques RLS sont posées et vérifiées, et une charte d'audit UI existe déjà. Le vrai point d'attention est ciblé : les écrans qui touchent à l'argent (facturation, paiements) ne sont pas testés, et rien n'empêche aujourd'hui un type-error de passer en production. Ce sont deux chantiers courts et peu coûteux à lancer dès ce sprint.